Deep multi-angle research — local-dependency obtaining mechanism (TMX-064)

Revision: 1 **Last modified:** 2026-06-28T00:00:00Z **Authority:** §11.4.150 (deep multi-angle research before closure), §11.4.8, §11.4.99 (latest-source cited) **Scope:** TMX-064 — git-ignored per-host local-dependency layer + cross-platform obtaining script (`scripts/obtain_local_deps.sh`); first consumer = jemalloc (amber has no host jemalloc

- no sudo; mistborn has it via brew but brew is off the non-interactive SSH PATH).

Why this research (§11.4.150)

Before closing TMX-064 (and the TMX-063 jemalloc gap it subsumes), confirm — from authoritative sources, ≥2 distinct angles — that (i) the chosen approach (obtain `libjemalloc.so.2` as a git-ignored local dep + make the binary/wrapper find it) is correct and standard, and (ii) there is no *bigger problem* we are unaware of. The hard project invariant (FACT-confirmed this cycle): **jemalloc MUST stay DYNAMIC** — `scripts/tests/61_*` T3 asserts `objdump -p | grep NEEDED.*libjemalloc`, and jemalloc is loaded via the wrapper's `LD_PRELOAD`. So static-linking is forbidden (would be a §11.4.1 fix-A-creates-B); the dependency must be *obtained* + *located*, not eliminated.

Angle 1 — Bundling a dynamic lib via rpath (DT_NEEDED resolution)

`-Wl,-rpath` / `patchelf` with `$ORIGIN` is the standard way to make an executable find a bundled `.so` without a system install. `$ORIGIN` resolves to the executable's own directory at run-time (single-quote it so the shell/`make` don't expand it). This satisfies the `DT_NEEDED libjemalloc` resolution (test-61 invariant preserved — jemalloc stays a dynamic dep).

- `patchelf --set-rpath` writes **RUNPATH** by default; add **`--force-rpath`** to write **RPATH**. RPATH takes precedence and (unlike RUNPATH) is NOT overridable by `LD_LIBRARY_PATH`.
- `make`/autoconf mangle a literal `$ORIGIN` in `LDFLAGS` (`$O` expansion) → set the rpath with **patchelf after the build**, or escape it — NEVER a raw `-Wl,-rpath,$ORIGIN` through `make`.

Angle 2 — LD_PRELOAD ignores rpath/ RUNPATH (THE gotcha)

man7 `ld.so(8)`: libraries named in **`LD_PRELOAD` are loaded BEFORE the normal search begins**, and preloads do **not** benefit from RPATH/RUNPATH the way regular `DT_NEEDED` deps do — so a preload pathname that is not a recognised token must be **absolute** (or a `$ORIGIN`/`$LIB`/`$PLATFORM` dynamic-string token, which `LD_PRELOAD` *does* understand) to be reliably found.

Consequence for us: the project preloads jemalloc, so the generated wrapper MUST hand `LD_PRELOAD` the **resolved ABSOLUTE path** to `libjemalloc.so.2` (host / brew-prefix / `.local-deps`). rpath alone is insufficient for the preload. → The mechanism needs **BOTH**: (a) patchelf rpath on the binary for the `DT_NEEDED` resolution, (b) the wrapper's `LD_PRELOAD`/`LD_LIBRARY_PATH` (Linux) / `DYLD_INSERT_LIBRARIES`+`DYLD_LIBRARY_PATH` (macOS) set to the resolved absolute path. This is the §11.4.111 "resolve-by-stable-identity, not ambient PATH" discipline applied to the runtime loader.

Angle 3 — jemalloc-specific + obtain method

jemalloc upstream (`INSTALL.md`) supports building from source with a custom `--prefix` (install into a git-ignored local prefix, no sudo) and `--with-jemalloc-prefix`/soname-suffix for coexistence. The standard production pattern for a host that can't install it system-wide is exactly `export LD_PRELOAD=/abs/path/to/libjemalloc.so.2` — i.e. obtain the `.so` locally + point the loader at it (precisely the operator's "git-ignored local dependency" directive). On a host with **no compiler** (amber routed to the containerised build for that reason), build/extract jemalloc inside the build container (glibc 2.35) and place the `.so` in the bind-mounted git-ignored prefix — amber's glibc 2.39 ≥ 2.35, so the container-built `.so` runs on the host.

No-bigger-problem verdict (§11.4.150(C))

No deeper defect surfaced. The only non-obvious trap — **LD_PRELOAD does not use rpath** — is real and is already handled by the design (absolute preload path + patchelf rpath for DT_NEEDED). The approach (obtain-locally + locate-via-rpath+absolute-preload, jemalloc stays dynamic) is the documented, standard one; it keeps test-61's `DT_NEEDED libjemalloc` invariant intact (no §11.4.1 regression) and needs no host install / no sudo on any platform.

Sources verified 2026-06-28

- ld.so(8) — Linux manual page (LD_PRELOAD load order; preload vs RPATH/RUNPATH; dynamic string tokens): https://man7.org/linux/man-pages/man8/ld.so.8.html
- patchelf(1) manpage (`--set-rpath`, `--force-rpath` RPATH-vs-RUNPATH): https://manpages.debian.org/unstable/patchelf/patchelf.1.en.html
- Creating relocatable Linux executables with RPATH `$ORIGIN` (single-quote / relocatable bundling): https://nehckl0.medium.com/creating-relocatable-linux-executables-by-setting-rpath-with-origin-45de573a2e98
- rpath — Wikipedia (RPATH vs RUNPATH precedence; LD_LIBRARY_PATH override): https://en.wikipedia.org/wiki/Rpath
- jemalloc INSTALL.md (build-from-source `--prefix`, prefix/soname options): https://github.com/jemalloc/jemalloc/blob/dev/INSTALL.md
- jemalloc production LD_PRELOAD pattern (obtain + absolute preload path): https://medium.com/@david.lowenfels/just-do-it-manually-3a7b4d151b44